

Sockets

Asignatura:

- Redes [TA048]
- Introducción a los sistemas distribuidos [75.43]

Presentado por *Sebastian Kraglievich*

31 de marzo, 2026



Aplicación de red

- Programa que se ejecuta en **diferentes sistemas finales** (hosts) y que **se comunican entre sí a través de una red** para intercambiar información y cumplir un objetivo.
- Una aplicación de red **implica al menos dos procesos** en máquinas distintas.
- Comunicación mediante protocolos específicos
- 2 tipos de aplicaciones de red (segun Kurose):
 - abierta (*open*)
 - propietarias

Todas las aplicaciones son aplicaciones de red? 🤔



Pero...

¿Cómo podemos comunicar dos procesos?
¿Existe alguna herramienta que podamos usar?

Por supuesto que sí, existen varias herramientas de *IPC (Inter Process Communication)*.
Y en la que nos concentraremos hoy se llama **socket**. 😊



¿Qué es un Socket?

- Un socket es una **abstracción provista por el sistema operativo**.
- Actúa como *interfaz entre un proceso de aplicación y la red*, permitiéndole enviar y recibir datos.
- En redes IP, un socket está asociado a un protocolo de transporte, que define cómo se envían los datos.

¿En qué capa del modelo TCP/IP actúa? ==> L4



¿Cómo se identifica un socket?

- Un socket queda identificado localmente en el SO por la terna:
 - **(protocolo, IP_local, puerto_local)**
- *socket != conexión*
- Esta info. la utiliza el SO para demultiplexar.
 - Depende de si usamos UDP o TCP.



¿Qué es un puerto?

- Número de 16-bit ==> 0 a 65535
- *well-known port numbers*: 0 al 1023
 - reservados para protocolos bien conocidos
- Si no asignamos manualmente ==> el SO asigna uno dinámicamente entre 1024 y 65535.
- El lado del servidor asigna puerto manualmente, el lado del cliente deja esto en manos del SO.

Número de puerto	Protocolo	Aplicación	Acrónimo
20	TCP	Protocolo de transferencia de archivos (datos)	FTP
21	TCP	Protocolo de transferencia de archivos (control)	FTP
22	TCP	Shell Seguro	SSH
23	TCP	Telnet	–
25	TCP	Protocolo simple de transferencia de correo (Simple Mail Transfer Protocol)	SMTP
53	UDP, TCP	Servicio de nombres de dominios	DNS
67	UDP	Protocolo de configuración dinámica de host (servidor)	DHCP
68	UDP	Protocolo de configuración dinámica de host (cliente)	DHCP
69	UDP	Protocolo de transferencia de archivos trivial	TFTP
80	TCP	Protocolo de transferencia de hipertexto	HTTP
110	TCP	Protocolo de oficina de correos versión 3 (Post Office Protocol version 3)	POP3
143	TCP	Protocolo de acceso a mensajes de Internet (Internet Message Access Protocol)	IMAP
161	UDP	Simple Network Management Protocol	SNMP
443	TCP	Protocolo seguro de transferencia de hipertexto	HTTPS

RFC 1700 ==> listado de numeros de puertos bien conocidos
Quedo obsoleta por RFC 3232

Capa de transporte (L4)

¿Cuál es la función principal que cumple la capa de transporte?

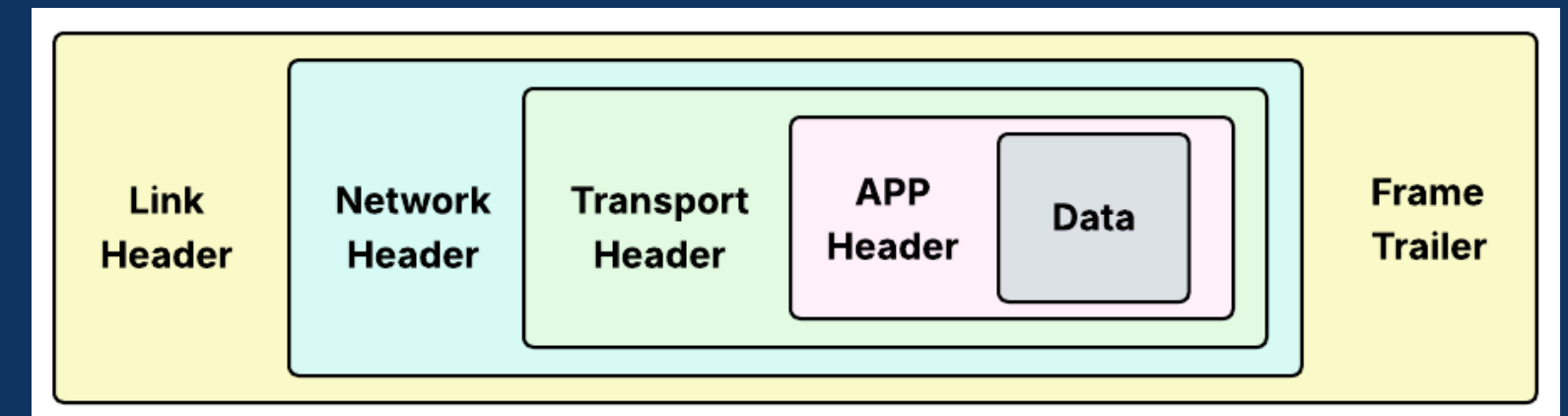
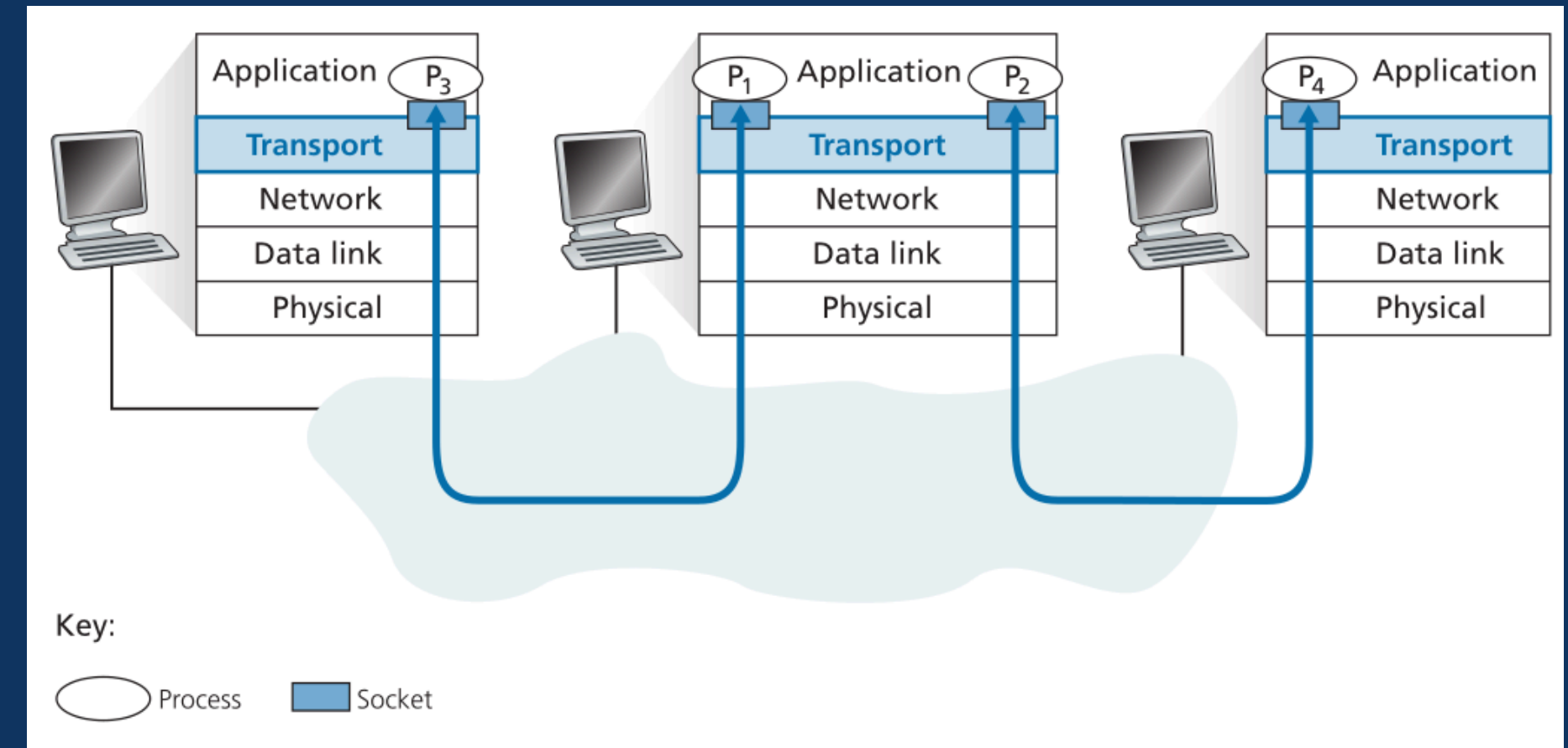
- Proporcionar **comunicación lógica extremo a extremo entre procesos que se ejecutan en distintos hosts.**

¿La entrega de datos a un proceso en el host destino siempre es confiable y segura?



Multiplexación y demultiplexación

- Multiplexación:
 - Reunir paquetes de datos (en *host origen*) provenientes de diferentes sockets y pasarlos a la capa de red.
- Demultiplexación:
 - Entregar datos de un segmento de capa de transporte al socket adecuado (en el *host destino*).
- UDP:
 - (IP_destino, puerto_destino)
- TCP:
 - (IP_origen, puerto_origen, IP_destino, puerto_destino)



Tipos de socket

Existen diferentes tipos de sockets:

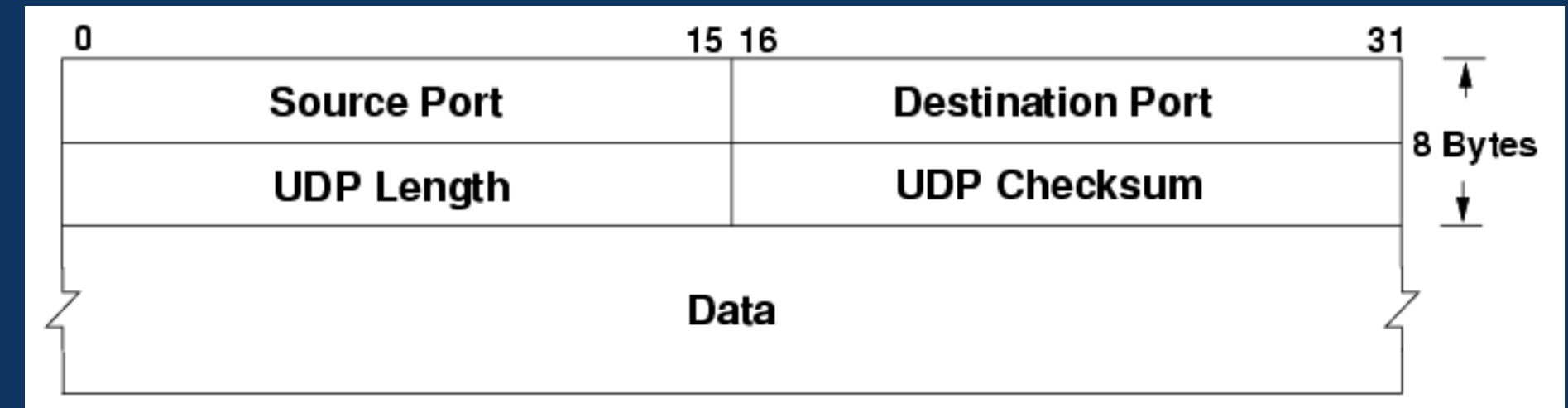
- Sockets UDP (*datagram sockets*)
- Sockets TCP (*stream sockets*)
- Sockets crudos o sin procesar (*raw sockets*)
- Sequenced packet sockets
 - Estan en desuso



UDP (User Datagram Protocol)

Características:

- connectionless
- stateless
- unreliable service
 - pérdida, duplicación, desorden



¿Que servicios ofrece?

- Process-to-process data delivery
 - Es confiable?
- Errors checking

¿Se puede lograr confiabilidad utilizando UDP?

Sí. Se debe implementar en la capa de aplicación.
Un ejemplo de esto es **QUIC**.

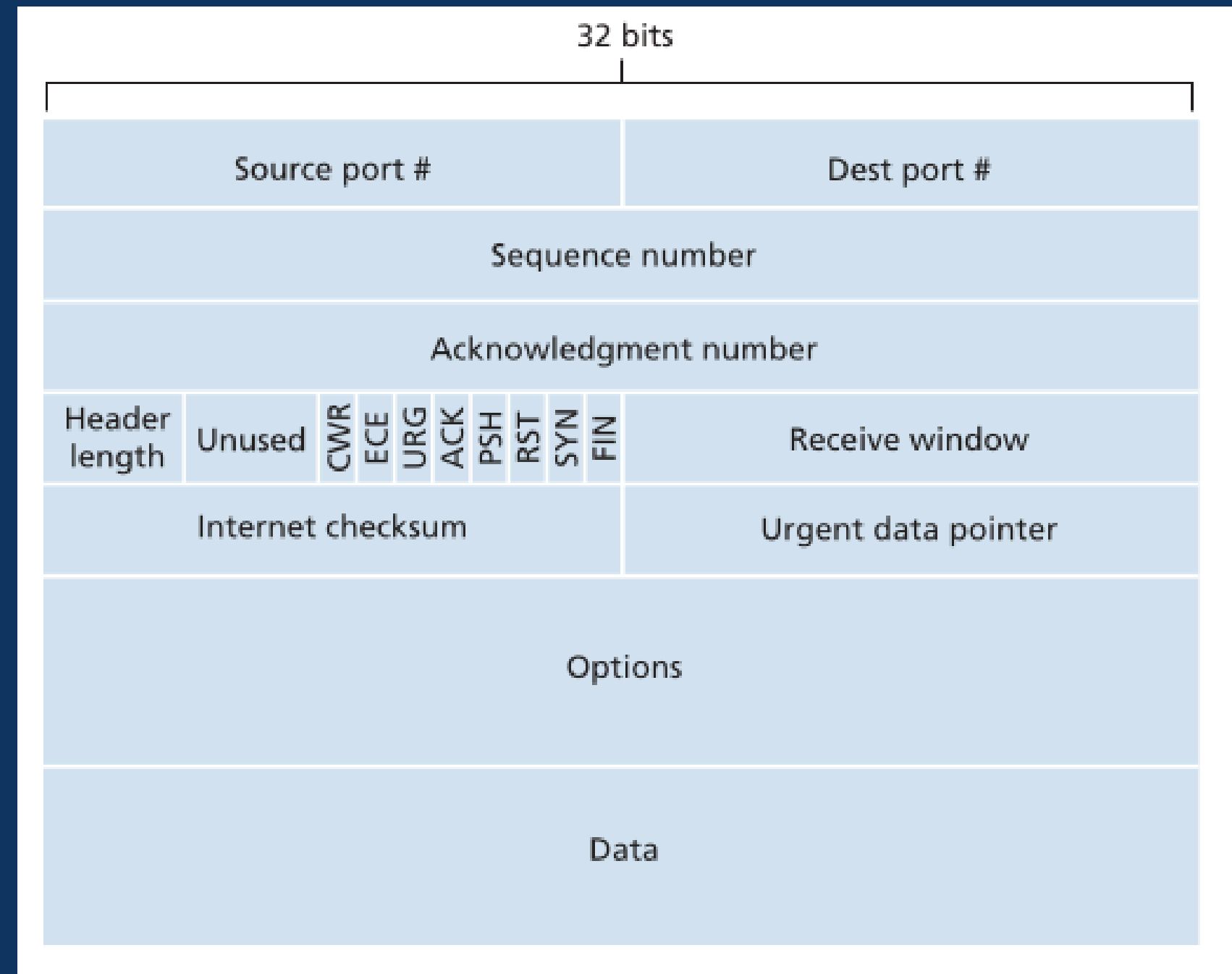
TCP (Transmission Control Protocol)

Características:

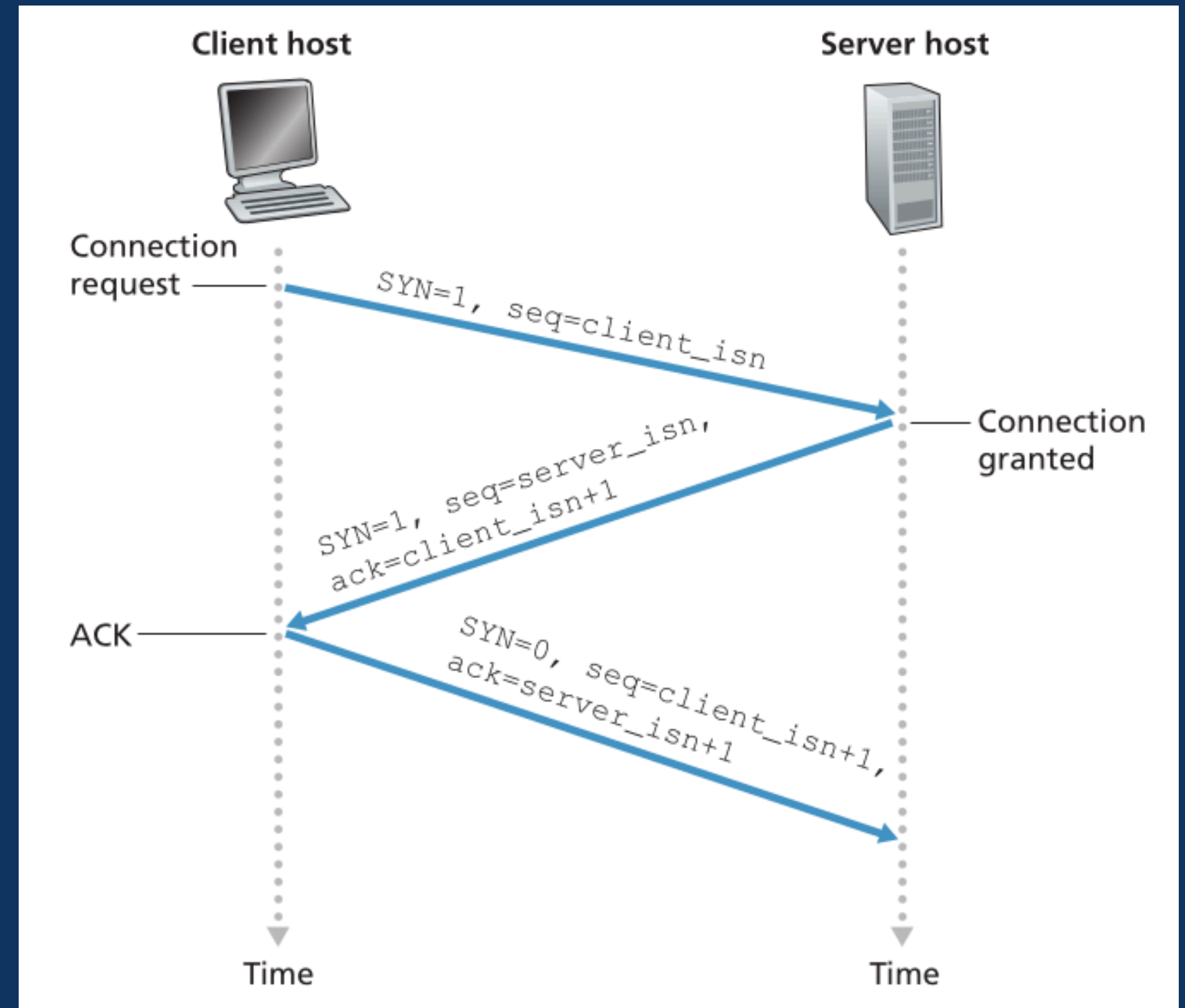
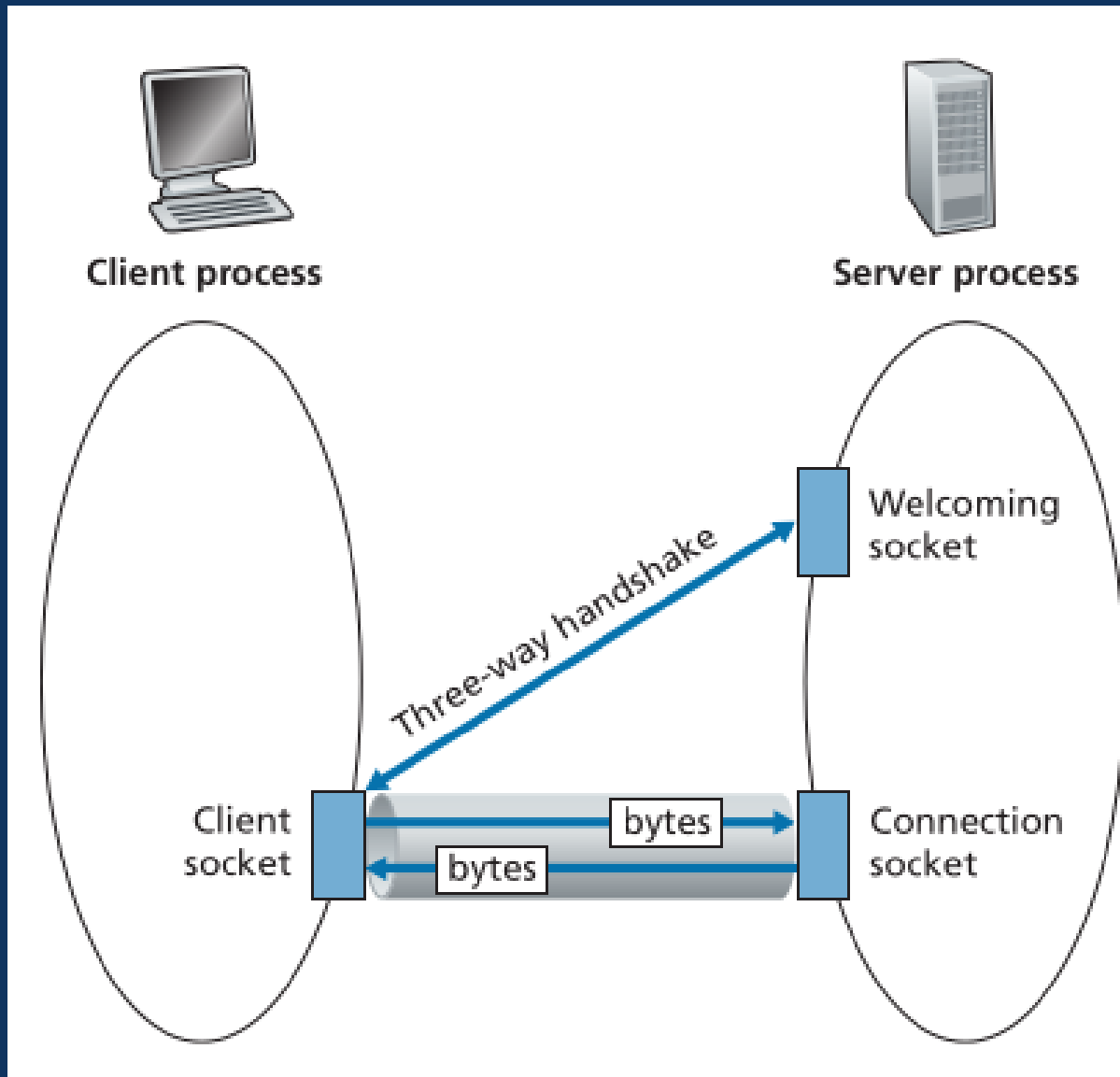
- connection-oriented
 - una conexión TCP:
 - full-duplex
 - point-to-point
- statefull
- reliable service

¿Que servicios ofrece?

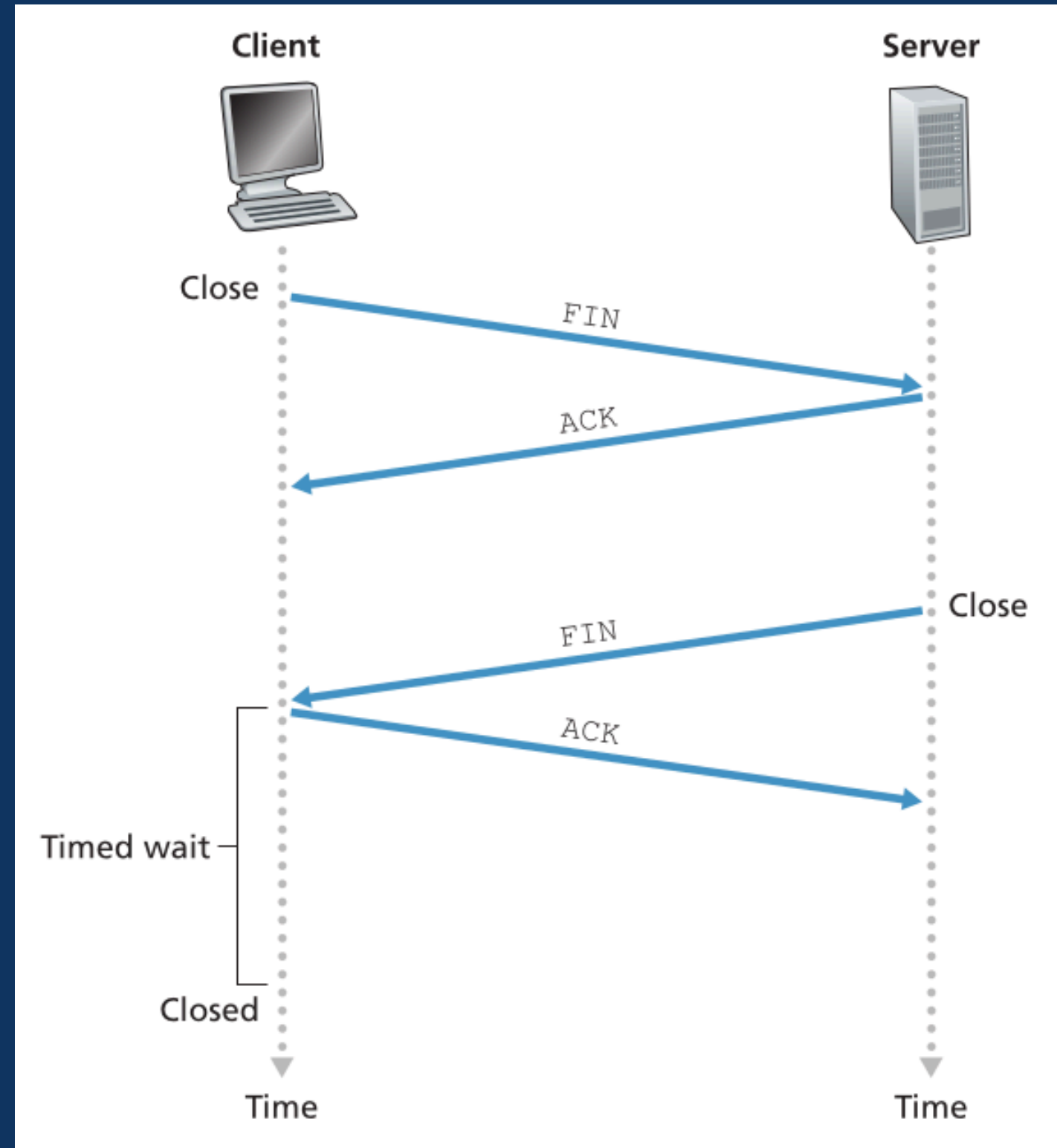
- Transferencia de datos confiable
 - mediante: control de flujo, seq_num, ack_num, temporizadores.
- Control de congestión



3-way handshake



4-way handshake



Demo time...

```
def greetings():  
    print("Hello, World!")  
  
def main():  
    greetings()  
  
if __name__ == "__main__":  
    main()
```

Propuesta para casa...

- Hacer servidor UDP multithreading
- Investigar ataque ***SYN flood*** y posibles contramedidas

```
def greetings():  
    print("Hello, World!")  
  
def main():  
    greetings()  
  
if __name__ == "__main__":  
    main()
```

¿Preguntas?

